CV Analyzer with Structured Outputs

Introduction to Generative AI and LLMs

Albert School of Business & Data

Project: CV Analyzer with Structured Outputs

Objective

In this task you will build, from scratch, a modular Streamlit application that uses an LLM to analyze a candidate's CV against a specific job description. The app will read a PDF, send its content to the model alongside the job requirements, and return a structured evaluation that the UI can render directly. The new core technique here is asking the model to produce a typed Pydantic object instead of free-form text.

> **Important.** This is a multi-step mini-project that pulls together everything you have seen so far: ChatPromptTemplate, LCEL, and modular code organization, plus a new piece — structured outputs with Pydantic. Take it one step at a time. If a step gives you trouble, isolate it in a small script and get it working before moving on.

What you will learn

- **Structured outputs**: use `with_structured_output()` to make the model return a typed Pydantic object.
- **Pydantic models**: describe the shape of your data with `BaseModel` and `Field(description=...)`. The descriptions act as a contract with the LLM.
- **Multi-variable prompts**: design a `ChatPromptTemplate` that consumes two inputs at once (CV text and job description).
- **Modular architecture**: split a real application into models, prompts, services, and UI.
- **Working with PDFs**: extract plain text from uploaded files with `PyPDF2`.

Project Architecture

You will organize the code into the following folder structure. Each module has a single responsibility:

```
cv_analyzer/
    app.py                  # entry point — launches the Streamlit UI
    models/
        cv_model.py         # Pydantic data model for the analysis result
    prompts/
        cv_prompts.py       # system + human message templates
    services/
        cv_evaluator.py     # builds the chain and runs the evaluation
        pdf_processor.py    # extracts text from the uploaded PDF
    ui/
        streamlit_ui.py     # all Streamlit components
```

Data flow: the UI receives a PDF and a job description, `pdf_processor` turns the PDF into text, `cv_evaluator` builds an LCEL chain made of `prompt | structured_llm`, and the typed result flows back to the UI for rendering.

Step-by-Step Implementation

1. Project Setup

Objective: create the folder structure shown above and an empty file for each module. Make sure your Python environment has the following packages installed:

- `streamlit`
- `pydantic`
- `PyPDF2`
- `langchain`, `langchain-core`, `langchain-google-genai`

Reflective question: why is it worth splitting a Streamlit app into multiple files instead of keeping everything in one `app.py`?

2. Define the Data Model (models/cv_model.py)

Objective: describe the shape of the analysis the model must produce. The LLM will be forced to match this schema, so the field names and descriptions are not decorative — they are instructions.

Your model must contain the following fields:

- `candidate_name` — full name of the candidate.
- `years_of_experience` — total years of relevant work experience (integer).
- `key_skills` — list of 5 to 7 skills most relevant to the role.
- `education` — highest education level and main specialization.
- `relevant_experience` — short summary of the experience most relevant to this specific role.
- `strengths` — 3 to 5 main strengths.
- `areas_for_improvement` — 2 to 4 areas the candidate could develop.
- `fit_percentage` — integer from 0 to 100 expressing fit to the role.

Starter skeleton:

```
from pydantic import BaseModel, Field

class CVAnalysis(BaseModel):
    """Data model for the complete analysis of a CV."""

    candidate_name: str = Field(description="Full name of the candidate, extracted from
the CV.")

    # Your turn — add the remaining fields described below.
    # Hint: use list[str] for collections, int for numeric fields,
    # and constrain the fit_percentage to the 0-100 range with ge=0, le=100.
```

Reflective question: why does writing a careful `description=` on each `Field` matter when the model is going to produce structured output?

3. Design the Prompts (prompts/cv_prompts.py)

Objective: write two message templates and combine them into a single `ChatPromptTemplate`.

Imports you will need:

```
from langchain_core.prompts import (
    ChatPromptTemplate,
    SystemMessagePromptTemplate,
    HumanMessagePromptTemplate,
)
```

What the system message should contain:

- Establish the persona — a senior recruiter with deep experience in tech hiring.
- State the evaluation criteria you care about (experience, technical skills, education, career coherence, growth potential).
- Set the tone — objective, professional, constructive.

What the human message should contain:

- Two template variables: `{job_description}` and `{cv_text}`.
- Clear instructions about what to extract, what to evaluate, and how to weight each factor when computing the fit percentage.

Starter skeleton:

```
# System prompt — define the recruiter persona and evaluation criteria
SYSTEM_PROMPT = SystemMessagePromptTemplate.from_template(
    """..."""  # Your turn
)

# Analysis prompt — accept the job description and the CV text as variables
ANALYSIS_PROMPT = HumanMessagePromptTemplate.from_template(
    """...
    {job_description}
    ...
    {cv_text}
    ..."""  # Your turn
)

# Combine both into a single ChatPromptTemplate
CHAT_PROMPT = ChatPromptTemplate.from_messages([
    SYSTEM_PROMPT,
    ANALYSIS_PROMPT,
])
```

Reflective question: what is gained by keeping the persona in the system message instead of putting everything into a single human message?

4. Build the Evaluator (services/cv_evaluator.py)

Objective: create the chain that turns the prompt plus inputs into a typed `CVAnalysis` object — no JSON parsing, no string post-processing.

Key new method: calling `llm.with_structured_output(CVAnalysis)` returns a new runnable that will deliver an instance of your Pydantic class. You then compose it with your prompt using LCEL exactly as before.

Starter skeleton:

```
from langchain_google_genai import ChatGoogleGenerativeAI
from models.cv_model import CVAnalysis
from prompts.cv_prompts import CHAT_PROMPT
```

```
def build_cv_evaluator():
    base_llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.2)

    # Key step: bind the Pydantic schema to the model
    structured_llm = base_llm.with_structured_output(CVAnalysis)

    # Compose the chain with LCEL
    chain = CHAT_PROMPT | structured_llm
    return chain

def evaluate_candidate(cv_text: str, job_description: str) -> CVAnalysis:
    chain = build_cv_evaluator()
    # Your turn — invoke the chain with both inputs and return the result.
    # Wrap the call in try/except so a failure doesn't crash the UI.
    ...
```

Reflective question: what concrete problems disappear once the model is forced to return a Pydantic object instead of free text?

5. Extract Text from the PDF (services/pdf_processor.py)

Objective: given a file uploaded through Streamlit, return its text content as a single string.

Starter skeleton:

```
import PyPDF2
from io import BytesIO

def extract_pdf_text(uploaded_file) -> str:
    """Reads an uploaded PDF and returns its concatenated text."""
    # Hints:
    # - Wrap uploaded_file.read() in BytesIO before passing to PyPDF2.PdfReader
    # - Iterate over pdf_reader.pages and call page.extract_text()
    # - Skip empty pages and join the rest with newlines
    # - Return a clear error string if nothing was extracted (image-only PDFs)
    ...
```

Reflective question: what should your function do if the PDF is a scanned image with no selectable text? How would the rest of the app find out?

6. Build the Streamlit UI (ui/streamlit_ui.py)

Objective: build a two-column layout. The left column collects inputs; the right column shows the analysis.

Inputs to collect:

- A PDF uploader: `st.file_uploader(..., type=["pdf"])`.
- A text area for the job description: `st.text_area(...)`.
- A primary action button: `st.button("Analyze candidate", type="primary")`.

Starter skeleton:

```
import streamlit as st
```

```
from models.cv_model import CVAnalysis
from services.pdf_processor import extract_pdf_text
from services.cv_evaluator import evaluate_candidate

def main():
    st.set_page_config(page_title="CV Evaluation System", layout="wide")
    st.title("CV Evaluation System")

    # Two columns: inputs on the left, results on the right
    col_input, col_result = st.columns([1, 1], gap="large")

    with col_input:
        # 1. File uploader for the PDF CV
        # 2. Text area for the job description
        # 3. "Analyze candidate" button
        ...

    with col_result:
        # If the button was clicked AND both inputs are present:
        #   - extract the PDF text
        #   - call evaluate_candidate(...)
        #   - render the resulting CVAnalysis object
        ...
```

Rendering the result: once you receive a `CVAnalysis` object, you have full type safety — the IDE knows every field. A suggested layout is shown below; the exact look is up to you.

```
def render_results(result: CVAnalysis):
    # Suggested layout:
    # - A big metric showing result.fit_percentage with a qualitative label
    #   (e.g. >=80 Excellent, >=60 Good, >=40 Average, <40 Low)
    # - The candidate's name, years of experience, and education
    # - The relevant_experience summary
    # - The key_skills as badges or success boxes
    # - Two side-by-side columns for strengths and areas_for_improvement
    # - A final hire / no-hire recommendation derived from fit_percentage
    ...
```

Reflective question: what should the UI do while the LLM is generating the analysis, so the user knows something is happening?

7. Wire It Together (app.py)

Objective: the entry point is tiny — it just calls the `main()` function from the UI module.

```
import streamlit as st
from ui.streamlit_ui import main

if __name__ == "__main__":
    main()
```

Run the app:

```
streamlit run app.py
```

Testing Your System

Try the app with at least three CVs and at least two different job descriptions:

- A CV that is a clearly strong match for the job description.
- A CV from a different field — observe how the fit percentage drops.
- A CV with little experience — observe how strengths and areas for improvement shift.

Verify that the structured object always parses correctly, even when the CV is messy or partially extracted from the PDF.

Final Reflections

- What advantages does `with_structured_output()` offer over asking the model for JSON in a prompt and parsing the response yourself?
- How would you handle CVs written in a language other than the one the prompts are written in?
- Where in this architecture would you plug in caching so that re-analyzing the same CV does not call the LLM twice?
- How would you add a second LLM call that, after the structured analysis, generates a one-paragraph hiring-committee memo addressed to the recruiter?